

# Audiobook Webapp — Tech Stack & Architecture Recommendations

## Project Overview

A local-first web application that scans your storage for audiobook files, enriches them with online metadata, and presents them in a Netflix-style browsing interface. The app will integrate with local AI models via LM Studio for metadata assistance and future recommendation features.

## 1. Development Environment

### Recommended: Docker Compose + VS Code Dev Containers

Since you enjoyed DDEV's workflow for Drupal, the closest equivalent for a Node/React project is **Docker Compose** paired with **VS Code Dev Containers**. This gives you the same benefits DDEV provided: one-command startup, consistent environments, isolated dependencies, and easy teardown.

**Why not DDEV itself?** DDEV is primarily designed for PHP-based stacks (Drupal, WordPress, Laravel). While it has some Node.js support, it's not optimized for a full Node/React development workflow with hot module replacement.

**Why Dev Containers?** VS Code Dev Containers let you define your entire development environment in a `devcontainer.json` and `docker-compose.yml`. When you open the project in VS Code, it automatically builds and starts all services, installs dependencies, and connects the editor to the running container — very similar to DDEV's `ddev start` experience.

### Suggested setup:

```
audiobook-webapp/
├── .devcontainer/
│   ├── devcontainer.json
│   └── Dockerfile
├── docker-compose.yml
├── Makefile                # make start, make stop, make logs, make scan
├── server/                 # Backend (Fastify API)
│   ├── src/
│   ├── package.json
│   └── tsconfig.json
├── client/                 # Frontend (React + Vite)
│   ├── src/
│   ├── package.json
│   └── tsconfig.json
└── README.md
```

A Makefile provides the DDEV-like command shortcuts:

- `make start` — spins up all Docker services
- `make stop` — stops all containers
- `make logs` — tails logs from all services
- `make scan` — triggers a library scan
- `make reset` — tears down and rebuilds from scratch

## 2. Tech Stack

### Frontend

| Component         | Recommendation      | Why                                                                                                                                                         |
|-------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework         | <b>React</b>        | Mature ecosystem, component-based, great for building Netflix-style grid UIs                                                                                |
| Build tool        | <b>Vite</b>         | Extremely fast HMR (hot module replacement), modern defaults, one-command scaffolding via <code>npm create vite@latest</code> ( <a href="#">Vite docs</a> ) |
| Language          | <b>TypeScript</b>   | Type safety catches bugs early, excellent IDE support in VS Code                                                                                            |
| Styling           | <b>Tailwind CSS</b> | Utility-first, rapid prototyping, consistent design system without writing custom CSS                                                                       |
| Component library | <b>shadcn/ui</b>    | Built on Radix UI + Tailwind, copy-paste components you own, no vendor lock-in                                                                              |

**Why this combination:** React + Vite + TypeScript is the current standard for modern web apps. Tailwind + shadcn/ui gives you polished, accessible components (cards, modals, dropdowns, navigation) out of the box — perfect for building a Netflix-style grid of audiobook covers with hover previews, detail modals, and playback controls.

### Backend

| Component       | Recommendation                         | Why                                                                                                                                                                                                                                                            |
|-----------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Runtime         | <b>Node.js</b>                         | Unified language with frontend, excellent audio/media library support                                                                                                                                                                                          |
| Framework       | <b>Fastify</b>                         | High performance, simple to get started, less boilerplate than NestJS. NestJS is an option if you want a more structured, Angular-like framework with dependency injection, but Fastify is lighter and sufficient for this project ( <a href="#">Fastify</a> ) |
| ORM             | <b>Prisma</b>                          | Type-safe database access, excellent DX, auto-generated migrations, supports SQLite and PostgreSQL. Start with SQLite for simplicity, migrate to PostgreSQL if needed later ( <a href="#">Prisma</a> )                                                         |
| Database        | <b>SQLite (MVP)</b>                    | Zero-config, file-based, perfect for a local-first single-user app. Be aware of WAL mode concurrency limits — if you add heavy background writing, consider PostgreSQL from the start                                                                          |
| Background jobs | <b>BullMQ + Redis</b><br>(when needed) | For async metadata fetching and file scanning. For the initial MVP, a simple worker function with a database job table is lower-friction. Add Redis/BullMQ once scanning becomes complex or needs to be resumable                                              |

## Audio/Media Processing

| Library                        | Purpose                                                                                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>music-metadata</b><br>(npm) | Parses ID3 tags, M4B metadata, cover art from audio files. Supports MP3, M4A, M4B, FLAC, OGG, WAV, and more ( <a href="#">npm: music-metadata</a> )                  |
| <b>FFmpeg / ffprobe</b>        | Extracts duration, chapter markers, codec info, and embedded cover art. Audiobookshelf uses FFmpeg as its media processing backbone. Install in the Docker container |
| <b>fluent-ffmpeg</b><br>(npm)  | Node.js wrapper for FFmpeg commands                                                                                                                                  |

**Important:** Support M4B files from the start. Audiobook libraries frequently use M4B (single-file audiobooks with chapter markers), not just MP3 folders.

## 3. Local AI Integration

### LM Studio Setup

LM Studio exposes a fully OpenAI-compatible REST API. Once you load a model and start the local server, it binds to `localhost:1234` by default and supports:

| Endpoint                          | Method | Purpose                                                |
|-----------------------------------|--------|--------------------------------------------------------|
| <code>/v1/models</code>           | GET    | List loaded models                                     |
| <code>/v1/chat/completions</code> | POST   | Chat completions (text)                                |
| <code>/v1/embeddings</code>       | POST   | Generate embeddings (for future recommendation system) |
| <code>/v1/completions</code>      | POST   | Text completions                                       |

You can use any OpenAI-compatible client library (JavaScript, Python) by pointing the base URL to `http://localhost:1234/v1` instead of `https://api.openai.com/v1`. No billing, no rate limits, no data leaving your machine ([LM Studio OpenAI Compatibility docs](#)).

### AI Integration Points

Wrap the LM Studio connection behind an **adapter interface** so you can swap to Ollama, OpenAI, or another provider later without touching business logic:

```
// server/src/ai/llm-adapter.ts
interface LLMAdapter {
  chat(messages: Message[]): Promise<string>;
  embeddings(text: string): Promise<number[]>;
}

class LMStudioAdapter implements LLMAdapter {
  private baseUrl = 'http://host.docker.internal:1234/v1';
  // ...
}
```

## Use AI for:

- Fuzzy matching of filenames/titles to metadata candidates (e.g., "audiobook\_01.mp3" → "The Lord of the Rings by Tolkien")
- Summarizing long book descriptions into short blurbs for the UI
- Extracting genres, themes, and mood tags from metadata
- Future: generating personalized recommendation explanations ("Because you listened to X, you might enjoy Y because...")

**Do NOT make AI required for basic functionality.** The app should work fully without the AI server running. AI enhances metadata matching and recommendations — it doesn't gate core features.

## 4. Metadata Pipeline

### Sources (in priority order)

1. **Embedded file tags** — Read via `music-metadata`. Many audiobooks already have title, author, narrator, and cover art embedded in ID3/M4B tags.
2. **Open Library API** — Free, no API key required for basic search. Search by title/author, get book covers, descriptions, and subject tags. Endpoints:  
`https://openlibrary.org/search.json?q=...` and  
`https://openlibrary.org/books/{isbn}.json` ([Open Library API docs](#))
3. **Google Books API** — Free with an API key, quota limits apply. Provides volume info, thumbnail images, categories, and descriptions.  
`https://www.googleapis.com/books/v1/volumes?q=...` ([Google Books API docs](#))
4. **Audnexus API** — Self-hostable, open-source (GPL-3.0), aggregates audiobook metadata from multiple sources into a unified API. Audiobookshelf uses it for chapter lookup. Requires MongoDB and Redis to self-host. Note: it relies on Audible-derived data and may be brittle or subject to terms-of-use considerations — treat as optional, not a foundation ([Audnexus GitHub](#))
5. **MusicBrainz / Cover Art Archive** — Useful for some releases and cover art, but not the primary audiobook metadata source. Open Library Covers and Google Books thumbnails are more relevant for audiobook covers.

## Pipeline Architecture

```
File on disk
  ↓
[1] Scanner: walk directory, extract embedded tags (music-metadata + ffprobe)
  ↓
[2] Parser: parse filename/folder structure → candidate title, author, narrator
  ↓
[3] Online lookup: query Open Library → Google Books → Audnexus (if configured)
  ↓
```

```
[4] AI matching (if enabled): LLM fuzzy-matches candidates to file metadata
    ↓
[5] Candidate store: save all candidates with confidence scores in DB
    ↓
[6] Manual review UI: user confirms or corrects the best match
    ↓
[7] Enriched metadata saved to app database
```

**Key principle:** Do not auto-write tags to media files by default. Store enriched metadata in the app's database first. Embedding metadata into audio files should be an explicit, manual action the user chooses to take later.

## 5. Database Schema (Conceptual)

```
Library
├── library_roots (paths to scan)
├── media_items
│   ├── title, author, narrator, description
│   ├── cover_image_path, cover_image_url
│   ├── release_date, genre, isbn/asin
│   ├── file_path, file_format (mp3/m4b)
│   ├── duration, chapters (JSON)
│   └── metadata_source, metadata_confidence
├── tracks (individual audio files for multi-file audiobooks)
├── contributors (authors, narrators, actors)
├── metadata_candidates (multiple matches with confidence)
├── user_overrides (manual corrections)
├── listening_history (progress, completed, timestamps)
└── scan_state (last scan time, files found, status)
```

## 6. Reference Project: Audiobookshelf

Before building from scratch, **install and run Audiobookshelf** to experience its UX and understand its feature set. It's the major existing benchmark for self-hosted audiobook apps.

**What Audiobookshelf does well:**

- Library scanning with auto-detection of updates (no manual rescan needed)
- Metadata and cover art lookup from multiple providers
- Chapter editor with chapter lookup via Audnexus
- Audio file tools: merge multiple files into single M4B, embed metadata
- Multi-user support with progress syncing
- Progressive Web App (PWA) with Chromecast support
- Companion Android/iOS apps

**Audiobookshelf's tech stack:**

- Node.js backend (67.5% JavaScript)
- Vue frontend (currently being rewritten to React)
- FFmpeg for media processing
- WebSocket for real-time communication
- Docker + Dev Containers for development
- Audnexus API for metadata

**Why build from scratch instead of using Audiobookshelf:**

- You want deep AI integration (LM Studio for matching, summarization, recommendations)
- You want to learn the full-stack development process
- You want full control over the UI/UX design
- Audiobookshelf's AI capabilities are limited/non-existent

**Recommendation:** Study Audiobookshelf's UX, schema design, and metadata approach. Use it as inspiration, but build your own for the AI integration and learning experience.

**Links:** [Audiobookshelf website](#) | [Audiobookshelf GitHub](#)

## 7. Recommended Development Phases

### Phase 1: Foundation (MVP)

- Set up Docker Compose + Dev Container with Node.js, client, and server
- Scaffold React + Vite + TypeScript frontend with Tailwind/shadcn
- Scaffold Fastify backend with Prisma + SQLite
- Build file scanner: walk directories, extract embedded tags via music-metadata + ffprobe
- Basic UI: grid of audiobook covers, detail view, playback

### Phase 2: Metadata Enrichment

- Integrate Open Library API for online metadata + cover art
- Integrate Google Books API as secondary source
- Build metadata candidate pipeline with confidence scoring
- Manual review/correction UI

### Phase 3: AI Integration

- Connect to LM Studio via OpenAI-compatible adapter
- AI-powered fuzzy matching (filename → title/author)
- Description summarization and genre extraction
- Store embeddings for future recommendations

## Phase 4: Recommendations (Secondary)

- Metadata-based recommendations (same author, genre, narrator)
- Listening history tracking and analysis
- AI-powered personalized recommendations with explanations
- Optional: add Audnexus for richer audiobook-specific metadata

## Phase 5: Polish

- M4B chapter support and chapter navigation
- Metadata embedding into files (manual action)
- PWA support for mobile
- Backup/restore

## 8. Things to Avoid

- **Don't auto-write tags to media files** — store in app DB first, make embedding a manual action
- **Don't rely solely on Audible/Audnexus** — it's unofficial and may be brittle; use it as one optional source
- **Don't make AI a hard dependency** — the app should function fully without LM Studio running
- **Don't skip M4B support** — audiobook libraries commonly use M4B, not just MP3
- **Don't over-engineer the backend** — start with Fastify + SQLite, add Redis/Postgres only when needed
- **Don't build without studying Audiobookshelf first** — install it, use it, learn from its UX and architecture

## 9. Summary

| Layer            | Technology                                                    |
|------------------|---------------------------------------------------------------|
| Dev environment  | Docker Compose + VS Code Dev Containers + Makefile            |
| Frontend         | React + Vite + TypeScript + Tailwind CSS + shadcn/ui          |
| Backend          | Node.js + Fastify                                             |
| Database         | SQLite via Prisma ORM (migrate to PostgreSQL if needed)       |
| Background jobs  | BullMQ + Redis (add when needed)                              |
| Audio processing | music-metadata (npm) + FFmpeg/ffprobe                         |
| Local AI         | LM Studio OpenAI-compatible API (localhost:1234)              |
| Metadata sources | File tags → Open Library → Google Books → Audnexus (optional) |

| Layer             | Technology                            |
|-------------------|---------------------------------------|
| Reference project | <u><a href="#">Audiobookshelf</a></u> |

This stack is modern, local-first, JavaScript/TypeScript throughout, and designed to support all your planned features including AI-powered recommendations — while keeping the development experience as smooth and DDEV-like as possible.